

Extra SQL Filters

GROUP BY is a filter to give output with *math related functions*.

- Let's say I wanted to COUNT how many Albums are tied to each unique producer?

First, let me explain WHY we want to "GROUP"

- This query, shows every Album tied to their respective title names.

```
SELECT Producer.producename, Album.albumtitle
FROM Producer
INNER JOIN Album
    ON Producer.producerid = Album.producerid
```

	producename	albumtitle
1	Max Martin	Midnight Rain
2	Max Martin	Sunrise Over
3	Max Martin	Starlight
4	Rick Rubin	Summer Pop
5	Rick Rubin	Dance Hits
6	Rick Rubin	Euphoria
7	Rick Rubin	Pop Revolution
8	George Martin	Classic Vibes

But, what if I want how many Album's a producer has producer? *Rather than which ones?*

```
SELECT Producer.producename, COUNT(*) AS AlbumCount
```

```
-- SELECT the name, COUNT(*) grabs EVERYTHING, do ANY
```

```
FROM Producer -- FROM the Producer Table
```

```
INNER JOIN Album -- ADD the Producer Table
```

```
ON Producer.producerid = Album.producerid -- JOIN the table via the keys
```

GROUP BY *Producer.producername* -- GROUP by the UNIQUE attribute which is also in the SELECT, i.e. the name.

	producername	AlbumCount
1	George Martin	1
2	Max Martin	3
3	Rick Rubin	4

Notice, how we used a **PINK** math related function, **GROUP BY** can use any of the functions taught in PART 3 of the lab.

- Since we can do ANY math function for **GROUP BY**'s, We can also **SUM** the fees gained from each producer.

```
SELECT Producer.producername, SUM(PERFORMANCES.fee) AS TotalFeesPaid
```

```
-- SELECT the name, SUM the fees, so we can get the total for all Albums.
```

```
FROM Producer -- FROM the Producer Table
```

```
INNER JOIN Album -- We need the album to match the albumid key in the next table.
```

```
ON Producer.producerid = Album.producerid
```

```
INNER JOIN PERFORMANCES -- PERFORMANCES is the table with the fees attribute
```

```
ON Album.albumid = PERFORMANCES.albumid -- So we need to INNER JOIN it
```

```
GROUP BY Producer.producername -- GROUP BY the name of the producer
```

	producername	TotalFeesPaid
1	George Martin	95000
2	Max Martin	530000
3	Rick Rubin	300000

Now, let's say I want to FILTER this *GROUP BY* result, with a **relational filter** $<, >, =, \leq, \geq, \neq$.

- In simple English, what if I want to only see Producers that had fees more than 100,000?

For that, you use **HAVING**, right after your *GROUP BY*

Take the query from before, look at the *SELECT* line's *MATH* function, and put it in the bottom like this.

- **HAVING** **SUM**(PERFORMANCES.fee) > 100000

```
SELECT Producer.producename, SUM(PERFORMANCES.fee) AS TotalFeesPaid
FROM Producer
INNER JOIN Album
    ON Producer.producerid = Album.producerid
INNER JOIN PERFORMANCES
    ON Album.albumid = PERFORMANCES.albumid
GROUP BY Producer.producename
HAVING SUM(PERFORMANCES.fee) > 100000
```

Outputs →

	producename	TotalFeesPaid
1	Max Martin	530000
2	Rick Rubin	300000

Now, you may notice; both **HAVING** and **WHERE** are filters we use to *decide which rows get outputted*.

We want to use **HAVING**, when you've already used *GROUP BY*, and you want to filter a **PINK** (math) function

We use **WHERE** for anything else.

That's it for BASIC SQL queries, and that's it for this lab. Next part covers INNER queries which is for comparing AGGREGATE (maths) for an ENTIRE table. And VIEWS (New tables, less info)